GCA Session 1 - Lab Report

Maxime Rombaut and Stijn Vandendriessche
Master of Electronics and ICT Engineering Technology

Abstract: In this report, we present our work on developing a CUDA-based application that can efficiently flip a large array of numbers using parallelization on the GPU. We explain the design and implementation details of our solution, evaluate the performance and compare it with a sequential CPU-based approach. We discuss the limitations and future directions of our work.

Keywords: CUDA, GPU, CPU, C++

1 Design

To write a program intended for execution on a Graphics Processing Unit (GPU), there are a few differences in comparison to a program designed for a Central Processing Unit (CPU). In this section we will compare the main building blocks of both programs.

CPU	GPU
allocate memory	allocate memory on host
fill the memory with data	fill the memory with data
	allocate memory on GPU
	copy data from host to GPU
execute sequentially	execute in parallel
	copy data from GPU to host

Table 1: *comparison between CPU and GPU program*

Since the goal is to flip large arrays, it is necessary to explicitly allocate the needed memory in both cases. However, when writing a program for the GPU, it is also necessary to allocate the same amount of memory on the GPU, copy the data to that memory and copy the data back to the host when the execution is done.

2 GPU execution

A program will always be initialized, and mainly be ran on the host (CPU). This means that the CPU is in control, and if we want to run code on the GPU, this needs to be explicitly described.

To do this, a kernel is written. This is a function that can be accessed and executed in parallel by the GPU. To run this code, a prefix is added to the function call that declares grid size and block size.

2.1 launch parameters

CUDA uses threads, blocks and grids. Every time a kernel is started, we need to configure these launch parameters.

There will be one whole **grid** for every **kernel**, that can be devided into **blocks**. The number of blocks is our first launch parameter: **gridsize**.

The **blocks** can be further devided into **threads**. These threads will do the actual operation defined in the kernel, so the amount of threads in total, is the amount of operations that can be performed. The amount of threads per block is our second launch parameter: **blocksize**.

2.2 Indexing

Since the threads are run in parallel, we need a clever way to do indexing. More specific, we need a way to get a unique index for every thread, so we can access our array and flip it using this index.

To do this, cuda uses three variables: **blockIdx, blockDim and threadIdx**. Respectively these are the block index in the grid, the amount of threads per block and the thread index in the block. All of these have x, y and z values, but since we only have a one-dimensional array we will only be using the x values. To calculate the index, the following formula is applied.

$$i = blockIdx.x * blockDim.x + threadIdx.x \quad (1)$$

3 Results

We ran the program, that can be found on our github, for arrays sizing from 1 000 000 up to 1 000 000 000 elements on a cpu and a gpu. The results are shown below.

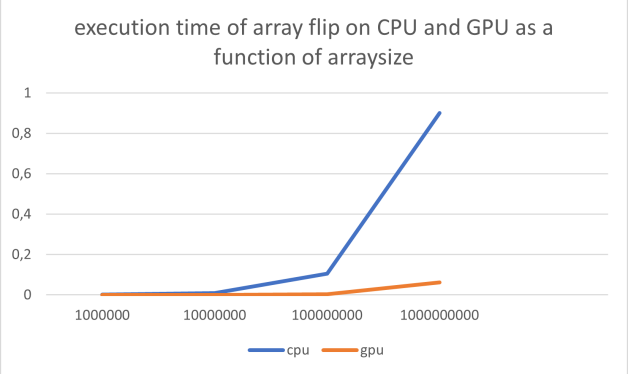


Figure 1: *execution time of CPU and GPU as a function of the size of the array*

Please note that the presented results only reflect the execution time on both the CPU and GPU. The time required to transfer data to and from the GPU is not included. This is a major overhead, making the total execution time for flipping an array on the CPU and GPU comparable.

4 Github link

`https://github.com/MaximeRombaut1412/geavanceerde_computerarchitectuur.git`